

MNLP Report Homework 2

Alessia Pontiggia
1892079

Gerardo Paladino
1918178

Mario Formato
2152143

1 Introduction

This report addresses the Sentence Splitter task using two approaches: encoder-based models (Sec.2.1) and a generative method (Sec.2.2). The experimental results are presented in Sec.3, followed by a comparative analysis in Sec.4. Finally, Sec. 6 reports and discusses the outcomes on the out-of-distribution (OOD) test set.

2 Methodology

2.1 Encoder Models: Bert and UmBerto

The encoder models tested are bert-base-italian-cased and it-umberto-focal-2. The adopted workflow is:

1. Tokens and corresponding labels are saved separately for both the training and validation sets, each in flattened lists. This separation ensures efficient preprocessing and simplifies subsequent chunking and alignment operations;
2. These flat sequences are segmented into fixed-length (64) chunks to fit the model's input size. If a sentence contains fewer tokens than the defined maximum length, padding tokens ("PAD") are added along with the special label -100. Assigning -100 as padding labels ensures that the corresponding tokens are ignored during both loss computation and evaluation, enhancing training efficiency;
3. Tokenization is performed using AutoTokenizer, which also aligns the labels precisely with the resulting tokens. This alignment is crucial in sequence labeling tasks, as it guarantees consistency between model input and supervised targets, even after subword tokenization;
4. The Berta model is trained on the full token sequences, treating the input as a continuous

stream of text. Training is performed on fixed-length windows (length 64) from this stream. When an End-of-Sentence (EOS) token is detected, the sentence is split; any remaining tokens are preserved and used as input for the next chunk. If no EOS is found, the entire sequence is buffered and retained until a complete segment is formed. This technique ensures context continuity and better exploitation of long dependencies. The UmBerto model uses for the classification task a custom token classification architecture built on top of a pretrained UmBERTo transformer followed by a dropout layer and two linear layers.

5. To address the significant class imbalance typical of this task, the Weighted Cross Entropy loss is employed for the berta model while UmBerto uses the focal loss. This enhances the model's sensitivity to underrepresented classes. Furthermore, dropout techniques are integrated during training to improve model generalization and prevent overfitting.

The full set of hyperparameters used in the experiments is reported in Table ?? and 4.

2.2 Generative Approach: Llama-3.2-1B and gemma-1.1-2b-it

The models employed are Llama-3.2-1B and gemma-1.1-2b-it.

1. The input text is divided into fixed-size chunks of 8 tokens, which are then appended to the predefined prompt. This number has been tested along with {8, 16, 32, 64}.
2. When the model is Llama-3.2-1B, the prompt is "Split the text with [EoS] tokens:", while when the model is gemma and "Segna con [EoS] ogni fine frase."

"Testo: {text}" "Output:" because of its italian optimization. The model input is a pair of the form (input_text, output_text) where the input_text is a concatenation of the prompt and the current chunk, the output_text is a concatenation of the current chunk and the tag [EoS] if the previous token has label 1.

3. Both the prompt (with the input tokens) and the target outputs are embedded using the Autotokenizer. The training lasts 5 epochs and the Cross Entropy Loss is used. Again the number of epochs is a result of finetuning.
4. After the model generates output sequences, the text is post-processed to extract predicted labels. These labels indicate whether a given token is immediately followed by an [EoS] token (label 1) or not (label 0).

3 Results

3.1 Encoder Models: Bert and UmBerta

The results on the dev set are reported in the Confusion Matrix (Fig. ??), Classification Report (Table ??) and loss plots ([??], [??]) for Bert Model and similarly for UmBerto Model, respectively in Fig. 7, Table 5, 5 and ???. As the figures and tables show, both models perform well: they are extremely effective at distinguishing between sentence boundaries and non-boundaries reaching overall accuracy and weighted F1-scores reported as 1.00. Bert and Umberto successfully identify all true sentence boundaries and Umberto introduces a small number of false positives. The training loss decreases rapidly at first and then plateaus, indicating stable convergence. The learning rate decreases linearly because of weight decay usage. The Bert evaluation loss decreases faster with respect to UmBerto evaluation loss.

3.2 Generative Models: Llama 3.2-1B and gemma-1.1-2b-it

The results on the dev set are reported in the Confusion Matrix (Fig. 9), Classification Report (Table 7) for Llama 3.2 Model and similarly for gemma-1.1-2b-it Model, respectively in Fig. 11, Table 9. As the figures and tables show, both models perform well: they achieve very high overall accuracy, with weighted F1-scores above 0.99. On one hand, Llama 3.2 tends to over-predict EOS class introducing more false positives, on the other hand Gemma model tries a more balanced approach resulting in a

better trade off between recall and precision. Since the models could cause errors when generating the output (instead of following the output template given as input, a new sentence was generated), a small subset of samples were not classified.

4 Comparative Analysis

The task of Sentence Splitting is hard for several reasons. The data is often unbalanced, punctuation always doesn't indicate sentence boundaries and syntax may be complex mainly in old texts as the one used for this homework. These challenges significantly influence the behavior of both encoder-based and generative models. As results show, the encoder approach likely suits better with respect to generative one. The reason behind this can be found in the theory basics: encoder models provide contextualized token-level representations ideal for classification using bidirectional context, while the generative models have a, hard, error-prone, step in between which consists in the sentence generation and then the classification module. The superiority of encoder-only models is especially evident in terms of false negatives. Encoders almost perfectly identify NOT-EOS tokens, whereas generative models frequently misclassify them. Consequently, encoder models achieve substantially higher precision, recall, and F1-scores, showing that they not only capture boundaries more accurately but also correlate better with ground-truth distributions.

5 Further Experiments

When dealing with poor results in the generative approach, other solutions have been tried and used:

- semantic similarity between words: for each token a score was added, computed as the similarity between that token and the words belonging to its same sentence in order to underline that the end-of-section tokens had a different, outlined, score with respect to the others.
- enrichment of the input with informative markers: for each token we added a marker underlying whether the token was a punctuation, closing or connective token.
- metadata-driven prompt approach: The prompt included contextual information about the input text such as the language, author,

genre, and century and a template that specifies the structure of the desired output

These approaches, combined or not, didn't help the models to get better results, underlying how difficult this task was for generative systems.

6 Additional task: OOD results

Results on the OOD test set are visible at Fig. 4 for the encoder approach and 8 and Fig. 10 and 12 for the generative models.

6.1 Encoder Models

Encoder models produce good results. The few errors they get occur when sentences are very short, with one or two words. In this case, they struggle to correctly classify the EoS, showing a bit more influence of the false negatives.

6.2 Generative Models

In the generative approach, most of errors are caused by false negatives since when models have hallucinations, then the tokens misrepresented are classified as 0 by default. Others occur when two punctuation symbols are subsequent or when a sentence contains only a few words (such as in the direct speech which is often used in this set) as stated previously.

6.3 Final Considerations

The OOD test set shows again the superiority of Encoder-based approach to this task. The common errors suggest a possible mitigation consisting in data augmentation to generate many short, single-word sentences with punctuation marked as EoS.

References

A Appendix

Hyperparameter	Value
Learning Rate (lr)	2×10^{-5}
Weight Decay	0.01
Batch Size	16
Frozen Layers	3
Epochs	10
hidden_dropout_prob	0.2
attention_probs_dropout_prob	0.2

Table 1: Training Bert hyperparameters

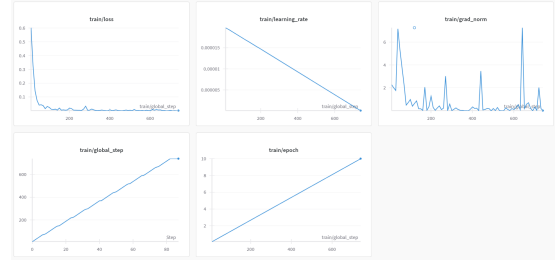


Figure 1: Bert Training Loss

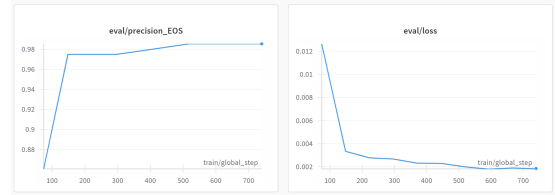


Figure 2: Bert Validation Loss

Class	Precision	Recall	F1-score
NOT_EOS	1.00	1.00	1.00
EOS	0.99	1.00	0.99
Acc			1.00
Macro	0.99	1.00	1.00
Weighted	1.00	1.00	1.00

Table 2: Classification report (Bert Encoder Model)

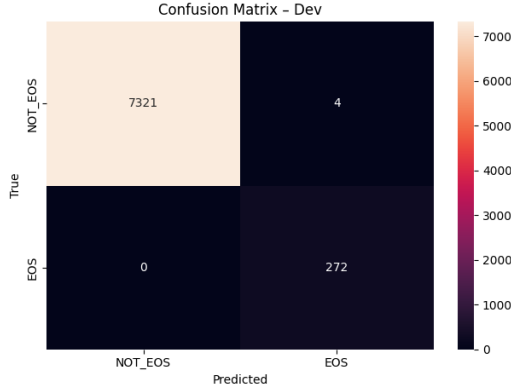


Figure 3: Bert Confusion Matrix

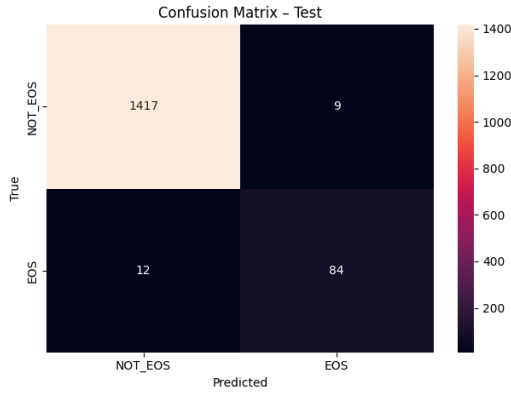


Figure 4: Bert Confusion Matrix on OOD Test Set

Class	Precision	Recall	F1-score
NOT_EOS	0.99	0.99	0.99
EOS	0.90	0.88	0.89
Acc			0.99
Macro	0.95	0.93	0.94
Weighted	0.99	0.99	0.99

Table 3: Classification report OOD (Bert Encoder Model)

Hyperparameter	Value
Learning Rate (lr)	2×10^{-5}
Weight Decay	0.01
Batch Size	16
Epochs	8
Frozen Layers	3
drop out	0.2
focal gamma	2.0
TARGET_EOS_RATIO	0.25

Table 4: Training UmBerto hyperparameters

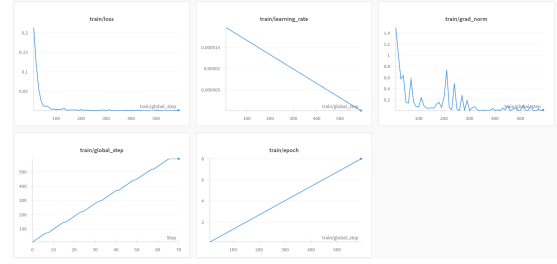


Figure 5: UmBerto Training Loss

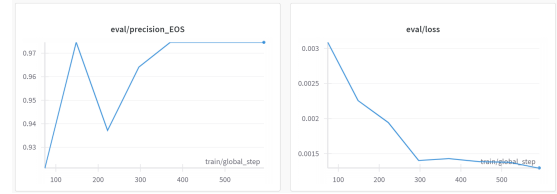


Figure 6: UmBerto Validation Loss

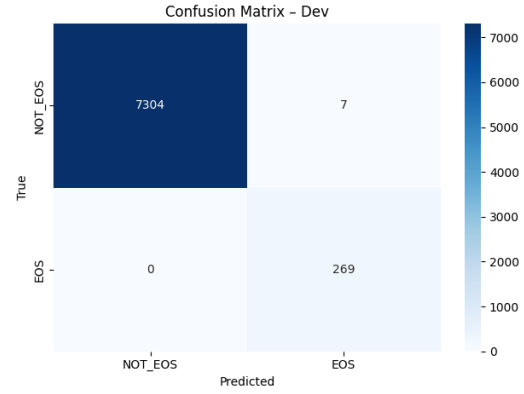


Figure 7: UmBerto Confusion Matrix

Class	Precision	Recall	F1-score
NOT_EOS	1.00	1.00	1.00
EOS	0.97	1.00	0.99
Acc			1.00
Macro	0.99	1.00	1.00
Weighted	1.00	1.00	1.00

Table 5: Classification report (UmBerto Encoder Model)

Class	Precision	Recall	F1-score
NOT_EOS	0.99	0.99	0.99
EOS	0.89	0.89	0.89
Acc			0.99
Macro	0.94	0.94	0.94
Weighted	0.99	0.99	0.99

Table 6: Classification report OOD (UmBerto Encoder Model)

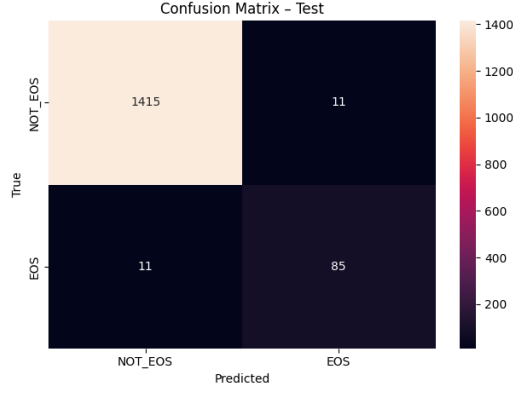


Figure 8: Umberto Confusion Matrix on OOD Test Set

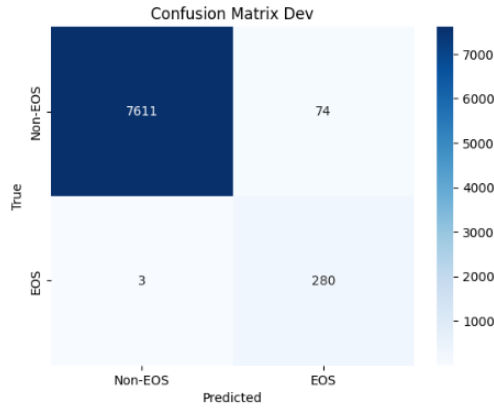


Figure 9: Llama 3.2 Confusion Matrix

Class	Precision	Recall	F1-score
NOT_EOS	0.9966	0.9904	0.9950
EOS	0.7910	0.9894	0.8791
Acc	0.9903		
Macro	0.8953	0.9899	0.9370
Weighted	0.9922	0.9903	0.9909

Table 7: Classification report (Llama 3.2 Model)

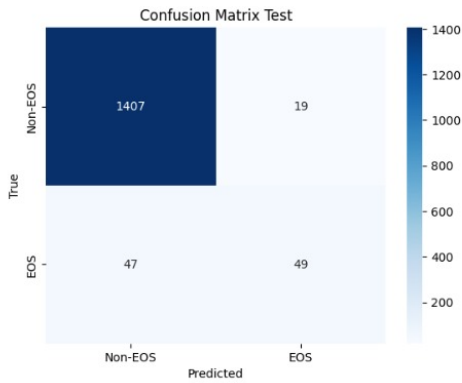


Figure 10: Llama 3.2 Confusion Matrix on OOD Test Set

Class	Precision	Recall	F1-score
NOT_EOS	0.9699	0.9712	0.9705
EOS	0.5591	0.5474	0.5532
Acc	0.9447		
Macro	0.7645	0.94	0.7619
Weighted	0.9442	0.9447	0.9445

Table 8: Classification report OOD (Llama 3.2 Model)

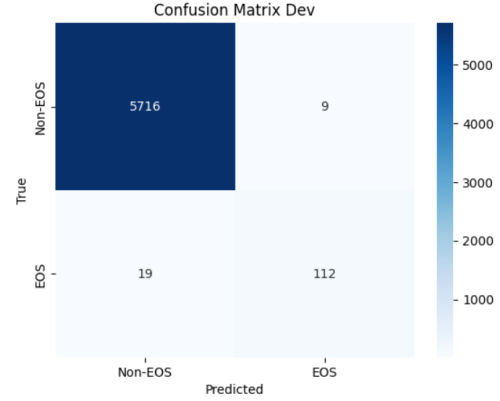


Figure 11: Gemma Confusion Matrix

Class	Precision	Recall	F1-score
NOT_EOS	0.9967	0.9984	0.9976
EOS	0.9256	0.8550	0.8889
Acc	0.9952		
Macro	0.9612	0.9267	0.9432
Weighted	0.9951	0.9952	0.9951

Table 9: Classification report (Gemma-1.1-2b-it)

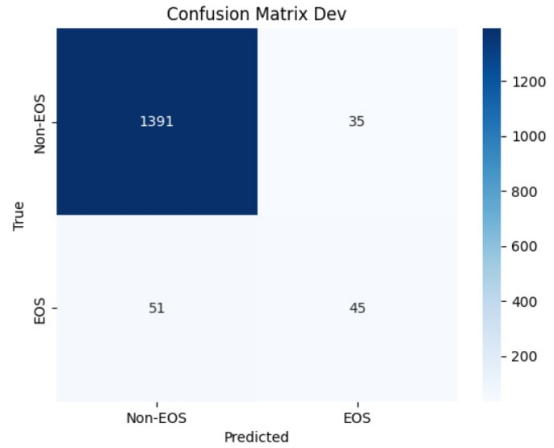


Figure 12: Gemma Confusion Matrix on OOD Test Set

Class	Precision	Recall	F1-score
NOT_EOS	0.9783	0.9811	0.9797
EOS	0.7033	0.6737	0.6882
Acc			0.9618
Macro	0.8408	0.8274	0.8339
Weighted	0.9611	0.9618	0.9615

Table 10: Classification report OOD (Gemma Model)